2. ИНТЕГРАЦИЯ МЕЖСЕТЕВОГО ЭКРАНА В КОРПОРАТИВНУЮ ВЫЧИСЛИТЕЛЬНУЮ СЕТЬ ГОЛОВНОГО ОФИСА

Ключевые слова: iptables, xtables-addons, SNAT, DNAT, masquerade, SSH, RDP, ip conntrack

Данная лабораторная работа посвящена основам разработки и проектирования корпоративной вычислительной сети с использованием программного межсетевого экрана iptables и интегрированными в него расширениями xtables-addons для пакетной фильтрации с трассировкой соединений. Рассматриваются различные варианты конфигурации, включая настройку NAT и MASQUERADE, фильтрации трафика, а также приемы противодействия активному анализу.

2.1 Введение в межсетевое экранирование

В настоящее время активного развития технологий актуальным остается фундаментальный вопрос обеспечения сетевой информационной безопасности. Злоумышленники используют все новые и новые методы получения несанкционированного доступа к корпоративной сети, когда как специалисты с другой стороны совершенствуют техники противодействия атакам. Важным этапом обеспечения безопасности сетевой инфраструктуры предприятия является межсетевое экранирование.

Межсетевой экран (англ. Firewall) – программное или аппаратно-программное решение, осуществляющее контроль и фильтрацию проходящего через него сетевого трафика. Под аппаратно-программными решениями понимается отдельное физическое устройство фильтрации трафика, когда как программные – ПО, устанавливаемое на устройство с той же целью.

Межсетевой экран зачастую требуется не только для обеспечения контроля за входящим и исходящим трафиком, но и для обеспечения безопасного базового взаимодействия сегментов сети. Например, на межсетевом экране также задаются правила NAT и MASQUERADE. На

межсетевом экране также устанавливаются общие правила взаимодействия с сегментом сети: например, откидываются все пакеты для иностранных IP-адресов или замедляются TCP-соединения при попытке подключиться по SSH:22, и так далее.

2.2 netfilter

Netfilter, основываясь на ранее данных определениях, является программным межсетевым экраном, подробнее – проект с открытым исходным кодом, разработанный для фильтрации пакетов и интегрированный в ядро Linux, начиная с версии 2.4.x.

Каждый пакет, дошедший до устройства, должен пройти этапы обработки netfilter, а именно проверку:

1) целостности;

2) принадлежности к определенному интерфейсу;

3) принадлежности к определенному процессу.

То есть, фактически, пакет проходит некоторое количество этапов определения дальнейшего пути перед тем, как попадет адресату. При этом, на каждом их них netfilter может:

1) продолжить передавать пакет (в рамках устройства);

2) откинуть пакет;

3) “захватить” – не уведомлять об откидывании пакета, при этом не продолжая его передачу;

4) поставить пакет в очередь;

5) повторить запрос.

В различных источниках его называют фреймворком, модулем ядра, межсетевым экраном и, по сути, все это является правдой: netfilter является модулем ядра Linux, который позволяет реализовывать различные сетевые операции, работает в пространстве ядра (kernel space), что

означает, что для взаимодействия с ним в пространстве пользователя (user space) также требуется программное обеспечение – iptables.

2.3 Введение в iptables

iptables – утилита командной строки Linux (интерфейс взаимодействия в пространстве пользователя) для управления netfilter. Рассмотрим фундаментальные основы работы с iptables.

iptables оперирует такими понятиями как таблицы и цепочки. Таблицей будем называть абстракцию для хранения цепочек. Цепочку же определим как упорядоченная последовательность правил. Слово упорядоченная оставлено не случайно: все правила выполняются исключительно друг за другом. То есть, если в одной цепочке первым правилом будет откидывать все пакеты, а вторым – передавать его некому процессу, весь трафик будет откидываться.

Существует пять встроенных цепочек: INPUT и OUTPUT, отвечающие за обработку входящего (адресатом которого является наше устройство) и исходящего трафика соответственно, FORWARD, применяемая к транзитным пакетам, а также PREROUTING и POSTROUTING, требуемые буквально для предобработки и постобработки пакетов (например, здесь могут изменяться заголовки пакетов для реализации NAT).

Данными цепочками оперируют следующие стандартные таблицы iptables: filter, raw, mangle и nat. Рассмотрим каждую из них.

Таблица filter используется как основная для фильтрации пакетов. Пакеты могут быть приняты и переданы дальнейшим обработчикам (ACCEPT), откинуты с уведомлением (REJECT) и без уведомления (DROP). При отбрасывании с уведомлением отправителю будет отправлен пакет, содержащий информацию о том, что его сообщение было доставлено, обработано, но не передано адресату (например, процессу). В

свою очередь DROP просто забывает о пакете, не предупреждая об этом адресанта. Это может быть полезно при обеспечении безопасности: вместо того, чтобы говорить потенциальному злоумышленнику, что порт на устройстве закрыт, мы можем откинуть пакет без уведомления, не раскрывая потенциально чувствительную информацию.

Таблица nat позволяет выполнить два основных действия: DNAT – изменение адреса и порта получателя пакета и SNAT – изменение адреса и порта источника. Таблица mangle используется для модификации самого пакета (например, изменения поля TTL или маркировки для маршрутизации). Таблица raw применяется для обработки пакетов на самом раннем этапе, еще до их попадания в трассировщик соединений (conntrack). Главная практическая задача raw — назначение цели NOTRACK, чтобы исключить высоконагруженный, доверенный трафик из механизма отслеживания состояний, тем самым значительно сэкономив ресурсы процессора.

Трассировщик iptables, он же механизм определения состояний, призван создать единый источник правды для отслеживания состояния установленных соединений. Наличие такого механизма позволяет разрабатывать более гибкие правила межсетевого экрана, например, не тратить время и ресурсы на обработку сегментов, отправленных в рамках установленного ранее соединения. Соединение может иметь четыре базовых состояния, изменяемого динамически. Так, первый пришедший пакет, содержащий, например, TCP SYN сегмент, обновит трассировщик записью NEW. Если сервер ответит положительно на рукопожатие, то состояние изменится на ESTABLISHED, установленное, а если пакет содержал некорректную информацию, то на INVALID. Существует также четвертое, одно из самых “хитрых”, как его называют в Iptables Tutorial 1.1.19 – RELATED. Соединение получает такое состояние, если оно было порождено другим ESTABLISHED соединением. Чтобы понять принцип

определения соединения как RELATED, рассмотрим принцип работы FTP (рисунок 1). Изначально происходит трехстороннее рукопожатие TCP на порту 21, называемое FTP control, фактически являющееся управляющим каналом. Далее, уже внутри установленного соединения, инициализируется еще одно, но уже на порту 20, по которому и происходит передача данных. Именно оно и определится как RELATED для первого ESTABLISHED соединения.

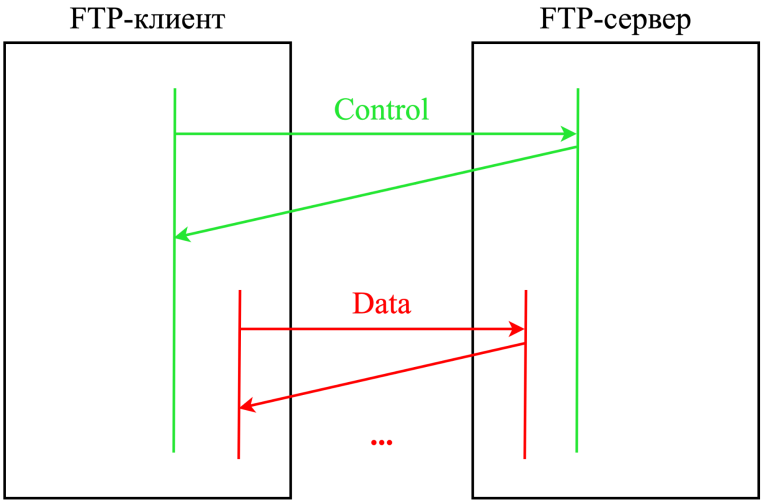


Рисунок 1 – Принцип работы FTP

Данные трассировщика можно посмотреть в procfs по пути /proc/net/ip_conntrack.

Итак, мы изучили весь терминологический аппарат, требуемый для понимания работы с iptables. Рассмотрим весь возможный путь приходящего в систему пакета (рисунок 2).

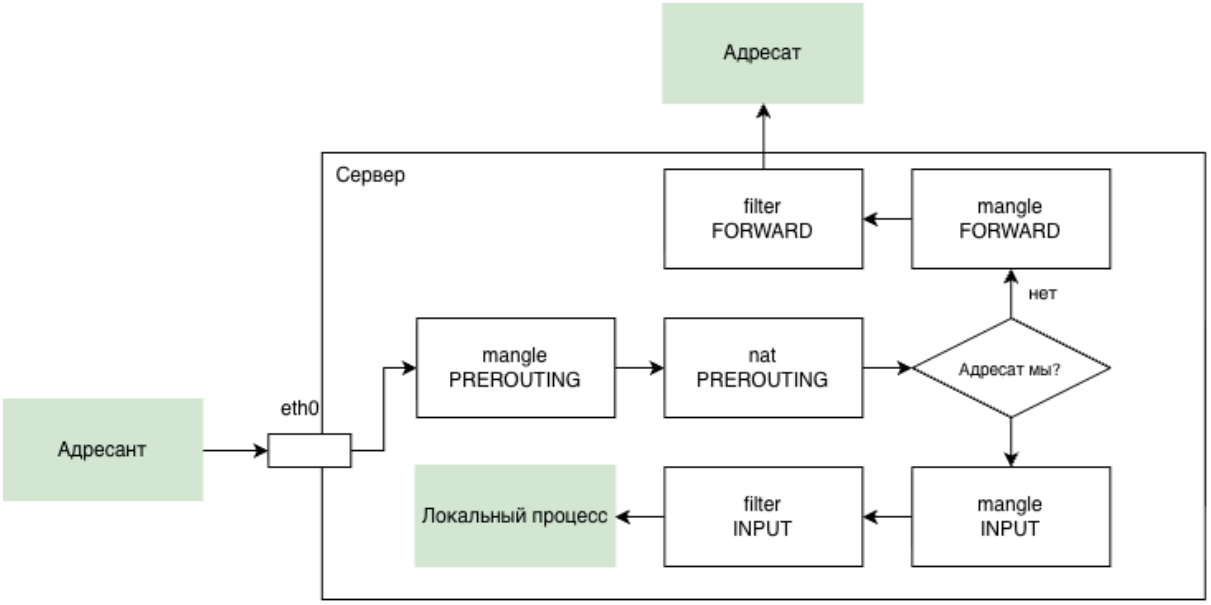


Рисунок 2 – Путь пакета от адресанта до локального процесса

Таким образом, приоритетно (не рассматривая таблицу raw) всегда выполняются правила mangle, после – filter. В случае, если пакет не является транзитным, приоритет выглядит следующим образом:

1) mangle;

2) nat;

3) filter.

Исторически утилита iptables являлась стандартом де-факто для управления netfilter во всех дистрибутивах Linux. Однако важно отметить, что в современных ОС (начиная с Debian 11, Ubuntu 22.04, RHEL 8) на смену устаревшему фреймворку пришел nftables. Он обеспечивает более высокую производительность, атомарность обновления правил и единый синтаксис для IPv4 и IPv6. При этом, для обеспечения обратной совместимости, в системах используется слой iptables-nft. Это означает, что привычные команды iptables продолжают работать, но под капотом правила транслируются и выполняются уже движком nftables.

Чтобы проверить установку утилит управления межсетевым экраном, выполните:

```
Bash
iptables --version
```

Если утилита установлена, вывод покажет версию, а наличие суффикса (nf_tables) подтвердит использование слоя совместимости. Если команда не найдена, установите пакет, используя пакетный менеджер (для Debian/Ubuntu):

```
Bash
sudo apt update && sudo apt install iptables
```

После этого вновь проверьте версию утилиты для подтверждения корректной установки.

2.4 Практические основы iptables

Рассмотрим хрестоматийный вариант реализации межсетевого экранирования в сети (рисунок 3). Есть две виртуальных машины: МЭ и ВМ. МЭ – виртуальная машина с дистрибутивом Linux и iptables. Имеет два интерфейса: один направленный в сторону интернета (WAN), второй – в локальную сеть (LAN). У ВМ единственный интерфейс, направленный в LAN.

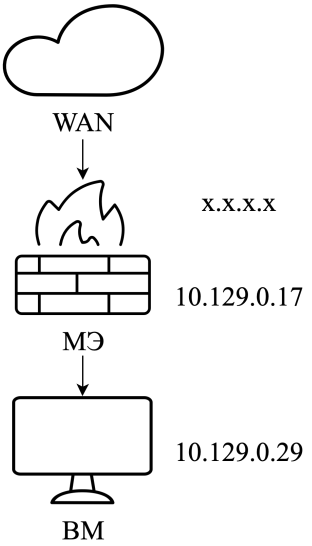


Рисунок 3 – Топология сети практической работы

Под x.x.x.x следует понимать IP-адрес на интерфейсе, который имеет доступ в WAN. Настроим NAT для ВМ на МЭ. Для того, чтобы мы могли передавать трафик другой машине требуется включить ip_forward:

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

Добавим в таблицу nat в цепочку POSTROUTING правило для маскарадинга:

```
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

И разрешим транзитное движение пакетов через eth1 (в LAN):

```
sudo iptables -A FORWARD -i eth1 -j ACCEPT
```

Также опционально можно настроить межсетевой экран как DNS сервера (пересылать все запросы, отправленные ::53, на 8.8.8.8:53):

```
sudo iptables -t nat -A PREROUTING -p tcp --dport 53 -j DNAT --to-destination 8.8.8.8:53
sudo iptables -t nat -A PREROUTING -p udp  --dport 53 -j DNAT --to-destination 8.8.8.8:53
sudo iptables -t nat -A POSTROUTING -j MASQUERADE
```

После этого требуется установить МЭ как шлюз для ВМ, что обеспечит доступ в WAN. Для последующих шагов исследования возможностей iptables установите с помощью вашего пакетного менеджера nginx, например, в Debian-based:

```
sudo apt install nginx
```

И запустите его, выполнив:

```
sudo systemctl start nginx
```

Проверить доступность извне можно, выполнив:

```
realnotema@10 ~ % curl <IP>
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is
successfully installed and
working. Further configuration is required.</p>
<p>For online documentation and support please refer
to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>
```

```
<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Где <IP> – адрес FW на интерфейсе eth0 (внешний). Попробуем запретить любой TCP и UDP трафик на порту http:80, будем откидывать пакеты без уведомления.

```
sudo iptables -I INPUT -p tcp --dport 80 -j DROP
sudo iptables -I INPUT -p udp --dport 80 -j DROP
```

Вновь попробуем получить страницу:

```
realnotema@10 ~ % curl 89.169.170.223 -k
curl: (28) Failed to connect to 89.169.170.223 port
80 after 75002 ms: Couldn't connect to server
```

Удалим все правила и вместо DROP будем выполнять REJECT при запросе на 80 порт:

```
sudo iptables -F
sudo iptables -I INPUT -p tcp --dport 80 -j REJECT
sudo iptables -I INPUT -p udp --dport 80 -j REJECT
```

Теперь мы понимаем, что сервер недоступен гораздо раньше, поскольку в этот раз не приходится дожидаться таймаута:

```
realnotema@10 ~ % curl 89.169.170.223 -k
curl: (7) Failed to connect to 89.169.170.223 port
80 after 112 ms: Couldn't connect to server
```

Помимо возможности непосредственного взаимодействия с пакетами iptables предоставляет интерфейс для написания правил логирования. Так, мы можем добавить правило, согласно которому все пришедшие на порт 80 пакеты будут попадать в журнал событий:

```
sudo iptables -F
# Создание цепочки с именем LOG_HTTP
sudo iptables -N LOG_HTTP
```

```
# Добавление в цепочку LOG_HTTP цели логирования с
# префиксом “LOG_HTTP: “
sudo iptables -A LOG_HTTP -j LOG --log-prefix
"LOG_HTTP: "
sudo iptables -A LOG_HTTP -j RETURN
sudo iptables -A INPUT -p tcp --dport 80 -j LOG_HTTP
```

После этого вновь отправим GET запрос на наш nginx и выведем журнал событий с поиском по “LOG_HTTP”:

```
yaa@fw:~$ sudo journalctl -k | grep "LOG_HTTP" |
head -n 1
Oct 01 19:41:57 fw kernel: LOG_HTTP: IN=eth0 OUT=
MAC=d0:0d:13:09:65:bd:00:00:5e:00:01:00:08:00
SRC=93.158.188.91 DST=10.129.0.18 LEN=64 TOS=0x00
PREC=0x60 TTL=53 ID=0 DF PROTO=TCP SPT=7651 DPT=80
WINDOW=65535 RES=0x00 CWR ECE SYN URGP=0
```

Также, для описания широчайших возможностей iptables рассмотрим еще один пример его использования: ограничение количества ssh сессий. Для этого добавим правило для протокола tcp и порта назначения 22, согласно которому запрещено устанавливать более одного соединения единовременно. Для этого выполним:

```
yaa@fw:~$ sudo iptables -A INPUT -p tcp --dport 22
-m connlimit --connlimit-above 2 -j REJECT
```

Из другой вкладки терминала попробуем выполнить подключение к SSH-серверу:

```
realnotema@10 ~ % ssh -l yaa 89.169.170.223
ssh: connect to host 89.169.170.223 port 22:
Connection refused
```

2.5 Практические основы xtables-addons

Несмотря на гибкость iptables, в реальной жизни для противодействия злоумышленникам приходится реализовывать более нестандартные методы защиты, которые на деле являются крайне распространенными в продакшн-средах. В этом случае на помощь приходит xtables-addons – расширение iptables, предоставляющая ряд модулей и возможностей над стандартной утилитой.

Например, xtables отлично подойдет для фильтрации трафика по стране происхождения, “захвата” соединения, установки rate limit – максимального числа соединений за определенное время.

Для установки модулей на Debian-based дистрибутивах требуется выполнить:

```
sudo apt install xtables-addons-common libtext-csv-xs-perl
```

Рассмотрим две цели: TARPIT и DELUDE. Оба инструменты предоставляют возможности введения злоумышленника в заблуждение относительно состояния вашей системы. Исследуем каждую цель по отдельности. Но перед этим удалим все созданные ранее цепочки iptables и просканируем нашу локальную машину утилитой nmap:

```
user@user-MacBook-Pro ~ % sudo nmap 89.169.170.223
Password:
Starting Nmap 7.98 ( https://nmap.org ) at 2025-10-01 23:49 +0300
Nmap scan report for 89.169.170.223
Host is up (0.0081s latency).
Not shown: 995 closed tcp ports (reset)
PORT    STATE    SERVICE
22/tcp  open     ssh
80/tcp  open     http
```

```
135/tcp filtered msrpc
139/tcp filtered netbios-ssn
593/tcp filtered http-rpc-epmap
Nmap done: 1 IP address (1 host up) scanned in 2.10
seconds
```

В результате сканирования получили информацию о наших портах. Каждый известный факт о вашей системе в руках злоумышленника становится оружием против вас, поэтому в интересах любого специалиста по информационной безопасности свести количество таковых к нулю.

Цель TARPIT захватывает TCP-соединение, фактически просто не давая злоумышленнику передавать информацию для исследования машины. Рассмотрим на практике. Добавим правило:

```
sudo iptables -A INPUT -p tcp --syn -m recent --name
maybe_scan --set
sudo iptables -A INPUT -p tcp --syn -m recent --name
maybe_scan --update --seconds 60 --hitcount 20 -j
TARPIT
```

Выполнив команды, мы создаем правило в цепочке INPUT для протокола TCP: если хост попытался установить соединение, он добавляется в set maybe_scan. Если же за 60 секунд таких попыток было больше предельного числа (в нашем случае – 20-ти) применяем TARPIT. Сканирование сервера с такими правилами вернет следующее:

```
user@user-MacBook-Pro ~ % sudo nmap 89.169.170.223
Password:
Starting Nmap 7.98 ( https://nmap.org ) at
2025-10-02 00:03 +0300
Nmap scan report for 89.169.170.223
Host is up (0.0085s latency).
PORT      STATE    SERVICE
```

```
1/tcp     open     tcpmux
3/tcp     open     compressnet
4/tcp     open     unknown
6/tcp     open     unknown
7/tcp     open     echo
9/tcp     open     discard
13/tcp    open     daytime
17/tcp    open     qotd
...
```

То есть все порты отвечают положительно на запрос соединения, что вынуждает сетевой сканер констатировать, что они открыты. Такое поведение межсетевого экрана исключит (в общем случае) возможность получения чувствительной информации о нашей системе.

При этом мы также можем использовать TARPIT для замедления соединения. Например, после добавления правила:

```
sudo iptables -I INPUT -p tcp --dport 80 -j TARPIT
```

Попытка подключиться по TELNET к порту 80 пройдет успешно, однако никакой информации получить не получится – сервер согласится выполнить соединение, но сразу же изменит размер окна (TCP WINDOW) на 0, что исключит возможность передачи любой информации.

Если же мы хотим создать видимость функционирования, например, netbus на порту 12345, то мы можем использовать вторую рассматриваемую цель – DELUDE. Добавим новое правило в цепочку INPUT:

```
sudo iptables -I INPUT -p tcp --dport 12345 -j DELUDE
```

При сканировании всего диапазона, порт 12345 начал классифицироваться NMAP как фильтруемый (filtered):

```
user@user-MacBook-Pro ~ % sudo nmap 89.169.170.223
```

```
Starting Nmap 7.98 ( https://nmap.org ) at
2025-10-02 00:06 +0300
Nmap scan report for 89.169.170.223
Host is up (0.0081s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE    SERVICE
22/tcp    open     ssh
80/tcp    open     http
135/tcp   filtered msrpc
139/tcp   filtered netbios-ssn
593/tcp   filtered http-rpc-epmap
12345/tcp filtered netbus
Nmap done: 1 IP address (1 host up) scanned in 2.12
seconds
```

Это также является крайне эффективным инструментом обеспечения информационной безопасности сервера.

Для фильтрации трафика по стране происхождения (что требуется в некоторых вариантах заданий) в xtables-addons применяется модуль `geoip`. Для его работы необходимо предварительно скачать и скомпилировать базу данных принадлежности IP-адресов странам (с помощью встроенных скриптов `xt_geoip_download` и `xt_geoip_build`). После размещения собранной базы в директории `/usr/share/xt_geoip/`, вы сможете создавать правила, разрешающие или блокирующие трафик по коду страны. Например, правило

```
sudo iptables -A INPUT -p tcp --dport 3389 -m geoip
! --src-cc RU -j DROP
```

отбросит все попытки подключения к порту RDP, если они исходят не с территории Российской Федерации.

2.6 port knocking

Port knocking (простукивание портов) – метод обеспечения безопасности сервера, при котором порт открывается исключительно в случае, если ранее клиент выполнил открывающую секретную последовательность обращений к неким портам.

Рассмотрим на практике. Установим knockd – наиболее простой (и поэтому наиболее подходящий для изучения) инструмент для реализации port knocking:

```
sudo apt install knockd
```

Теперь, изучив файл /etc/knockd.conf мы можем увидеть стандартную последовательность портов, которые нужно “простукать” для того, чтобы открылись порты ssh:22 и https:443.

```
yaa@fw:~$ cat /etc/knockd.conf
[options]
    UseSyslog
[openSSH]
    sequence    = 7000,8000,9000
    seq_timeout = 5
    command     = /sbin/iptables -A INPUT -s %IP% -p
tcp --dport 22 -j ACCEPT
    tcpflags    = syn
[closeSSH]
    sequence    = 9000,8000,7000
    seq_timeout = 5
    command     = /sbin/iptables -D INPUT -s %IP% -p
tcp --dport 22 -j ACCEPT
    tcpflags    = syn
[openHTTPS]
    sequence    = 12345,54321,24680,13579
```

```
    seq_timeout = 5
    command       = /usr/local/sbin/knock_add -i -c
INPUT -p tcp -d 443 -f %IP%
    tcpflags    = syn
```

С другой машины “простукаем” порты 7000 8000 9000:

```
yaa@site:~$ knock 89.169.170.223 7000 8000 9000
```

Вернемся на fw и выведем правила. Заметим, что после простукивание появилось новое, открывающее ssh:22:

```
sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT      tcp  --  89.169.184.55        anywhere
tcp dpt:ssh
```

После выполнения knock для последовательности обратной – 9000 8000 7000 правило удаляется.

Как и в криптографии, основой безопасности сервера с установленным механизмом простукивания портов, является не секретность существования такового, а последовательность портов, требуемых для открытия доступа к целевому ресурсу. Также стоит отметить, что из сервисов knockd и иных реализаций допускается разработка цепочки зависимых серверов (рисунок 4). Однако, мнение о том, что port knocking является основным или же даже одним из методов, способных обезопасить удаленный хост, попросту обманчиво.

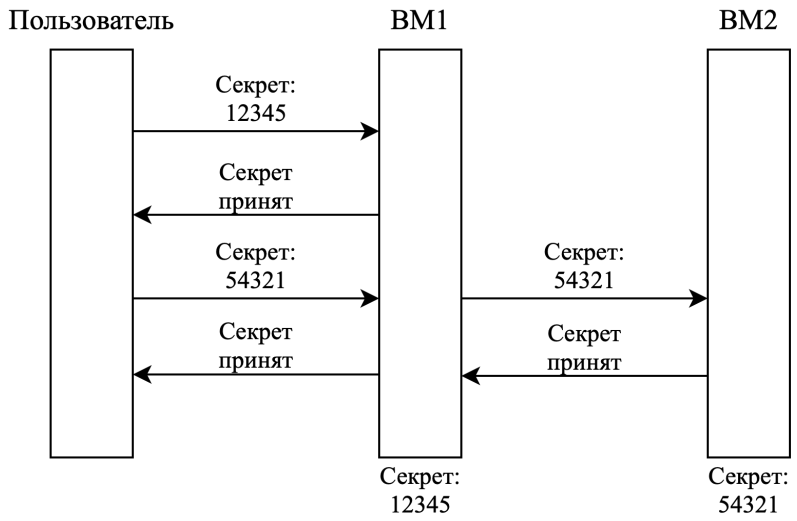


Рисунок 4 – Типовая схема цепочки простукивания портов

Основная идея информационной безопасности заключается в том, что панацеи, лекарства от любого злоумышленника, попросту не существует и существовать по определению не может. По этой причине многие системные администраторы не рассматривают данный механизм для встраивания в свою инфраструктуру.

2.7 Протоколы удаленного доступа

В зависимости от системы, для получения удаленного доступа используются различные протоколы. Так, для проприетарной ОС Windows существует такой же проприетарный протокол RDP (англ. Remote Desktop Protocol, протокол удаленного рабочего стола), а для Linux-систем наиболее используемым является SSH. Важным моментом остается то, что приведенные в этом разделе протоколы не являются исчерпывающими, а выбор зависит исключительно от выполняемой задачи.

Например, для корректной работы системы управления конфигурациями Ansible на Windows-хостах можно как развернуть

SSH-сервер, что зачастую является нетривиальной задачей из-за особенностей ОС, так и открыть встроенный WinRM listener (англ. Windows Remote Management, протокол удаленного управления Windows). На Linux в свою очередь в частных случаях может потребоваться подключения к графическому интерфейсу, из-за чего придется разобраться с RFB (англ. Remote FrameBuffer).

Однако в рамках настоящего курса частности упускаются и рассматриваются технологии, которые используются на постоянной основе почти везде.

SSH (англ. Secure Shell) – протокол для безопасного удаленного администрирования хостов, чаще используемый в Linux. Для его использования требуется сгенерировать два ключа: открытый и закрытый; первый требуется для шифрования, а второй – для расшифрования. Стандартно сервер функционирует на 22-ом порту, однако это можно изменить в конфигурационном файле /etc/ssh/sshd_config. Там же можно гибко настроить систему: запретить (разрешить только по ключу) вход в оболочку пользователя root, ограничить вход в оболочку любого другого пользователя и т.д.

RDP в свою очередь проприетарный протокол Microsoft для удаленного подключения к Windows-хостам. Стандартно функционирует на порту 3389, поддерживает NLA (англ. Network Level Authentication), аутентификацию на сетевом уровне, ограничение доступа RDP через VPN или некоторые IP-адреса и т.д. Рассмотрим установку на практике. Все представленные далее команды были протестированы на Windows Server 2022 и являются базовыми, что означает, что для конфигурации на более новой версии Windows предполагается прохождение тех же шагов с возможными небольшими изменениями.

Также для тестирования нам потребуется вторая виртуальная машина, с которой будем осуществлять подключение и тестировать

обновления конфигурации. Windows Defender рекомендуется полностью отключить на обеих машинах. Для начала попробуем подключиться к “чистому” Windows Server по RDP. Для этого выполним Win + R и введем mstsc.exe. При попытке подключиться RDP вернет ожидаемую ошибку недостижимости (рисунок 5).

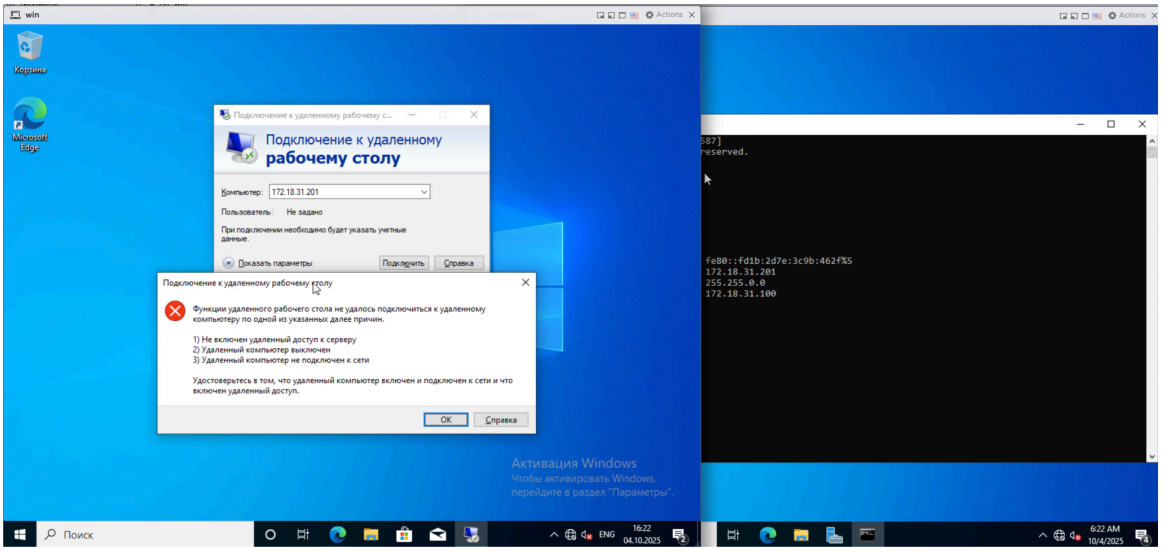


Рисунок 5 – Попытка подключения без включенного RDP-сервера

Разрешим подключение по RDP к нашей машине. Для этого в поиске Windows найдем SystemPropertiesRemote.exe (рисунок 6) и разрешим удаленно подключение (рисунок 7).



Рисунок 6 – Приложение для управления RDP-сервером

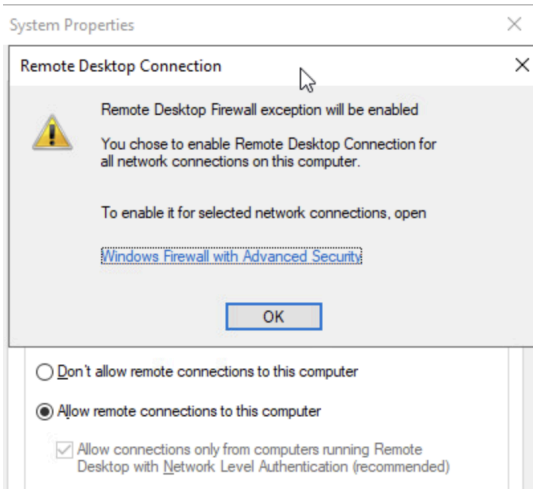


Рисунок 7 – Разрешение удаленного подключения к компьютеру

После разрешения появится уведомление о том, что появятся исключения Windows Defender, разрешающие подключение по RDP. После нажатия “Ok” и “Apply” вновь попробуем подключиться с другого хоста (рисунок 8). Обратим внимание, что NLA включен автоматически.

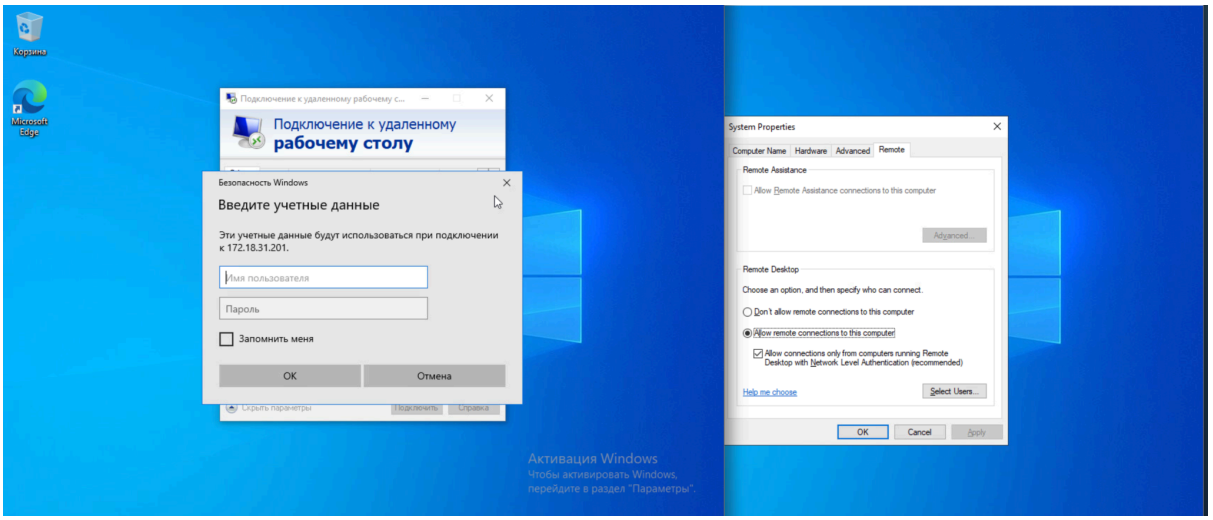


Рисунок 8 – Попытка подключения

При введении учетных данных локального администратора требуется согласиться подключиться к ресурсу с недоверенным сертификатом. После этого экран Windows Server заблокируется и вы получите доступ к системе (рисунок 9).

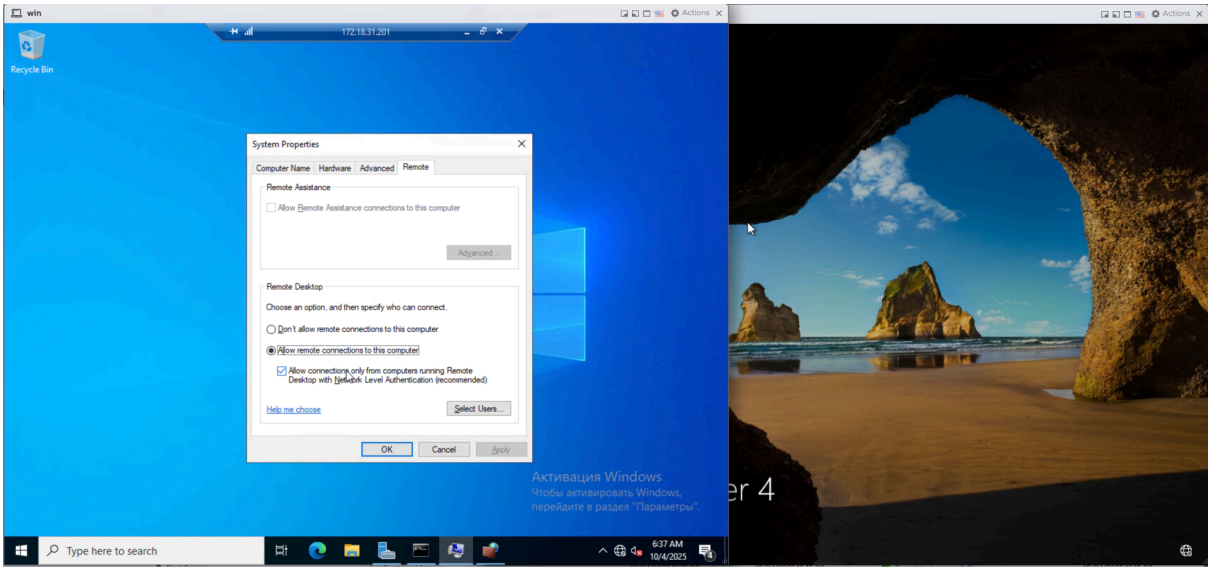


Рисунок 9 – Подключенное устройство

По стандарту Windows использует RDP Security Layer – встроенное в протокол шифрования. Чтобы изменить его на TLS, в поиске введем gpedit (рисунок 10).



Рисунок 10 – Приложение для управления групповыми политиками

Далее перейдите в Конфигурация компьютера → Административные шаблоны → Компоненты Windows → Службы удаленных рабочих столов → Узел сеансов удаленных рабочих столов → Безопасность. Здесь выберите “Требовать использование определённого уровня безопасности для удаленных подключений (RDP)”, нажмите на “Enable” и выберите “SSL” (рисунок 11).

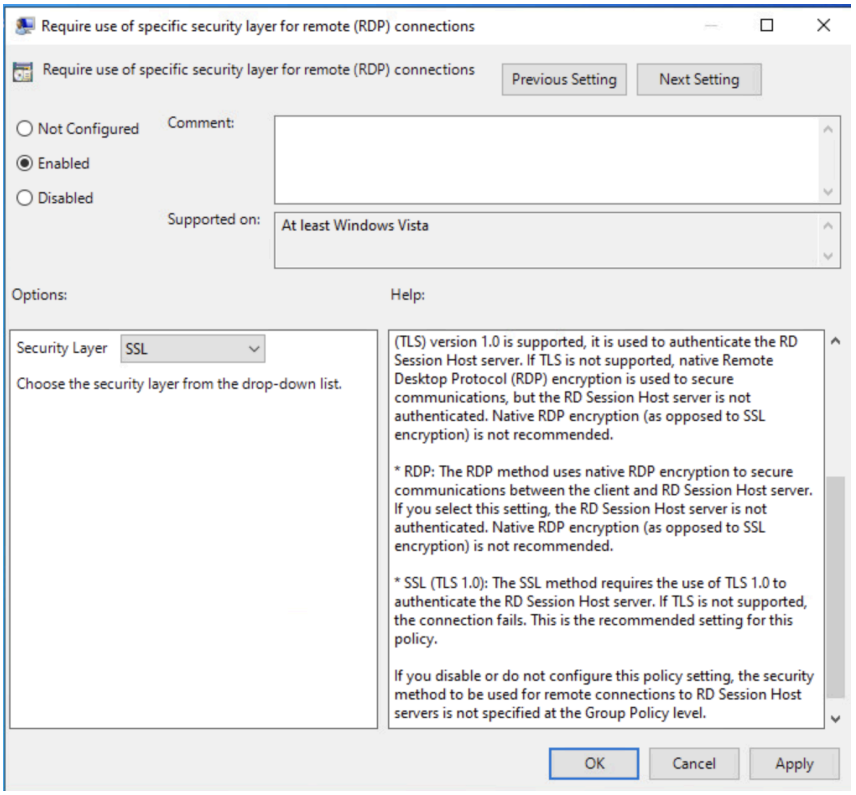


Рисунок 11 – Групповая политика для перехода на RDP через SSL (TLS)

После повторной попытки подключиться вновь должно появиться предупреждение о невозможности проверки подлинности сертификата. Чтобы убедиться, что мы используем SSL, нажмем на “Посмотреть сертификат” (рисунок 12).

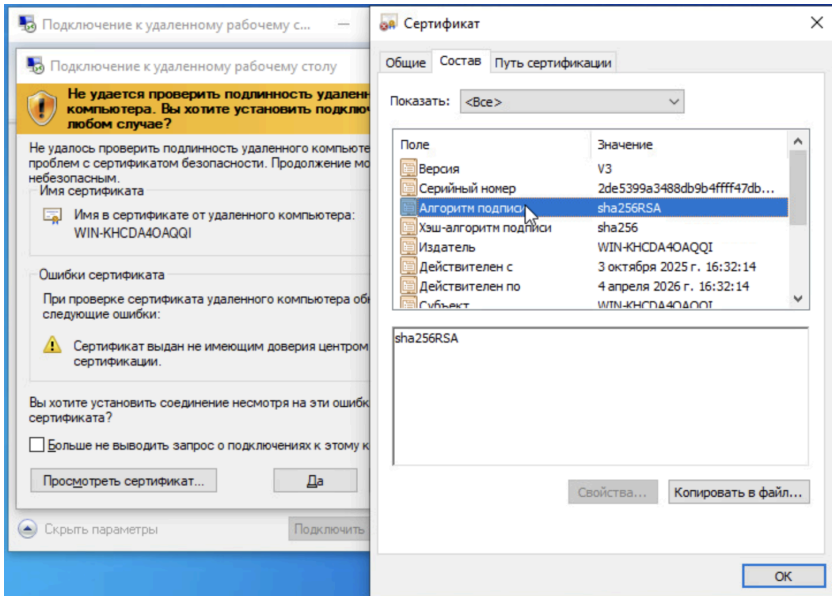


Рисунок 12 – Попытка подключения при включении SSL (TSL)

Важное примечание: В описании групповой политики (Рисунок 11) упоминается поддержка устаревшего протокола TLS 1.0. В современных корпоративных сетях использование TLS версий 1.0 и 1.1 строго запрещено из-за наличия известных уязвимостей. Системный администратор обязан дополнительно через реестр Windows или параметры шифрования SCHANNEL форсировать использование актуальных протоколов TLS 1.2 или TLS 1.3 для всех удаленных подключений.

Для дальнейшей работы создадим двух пользователей: Artem и Ivan и дадим им возможность входа через RDP. Для этого вернемся в приложение настройки RDP и, нажав на “Add”, добавим туда наши аккаунты (рисунок 13).

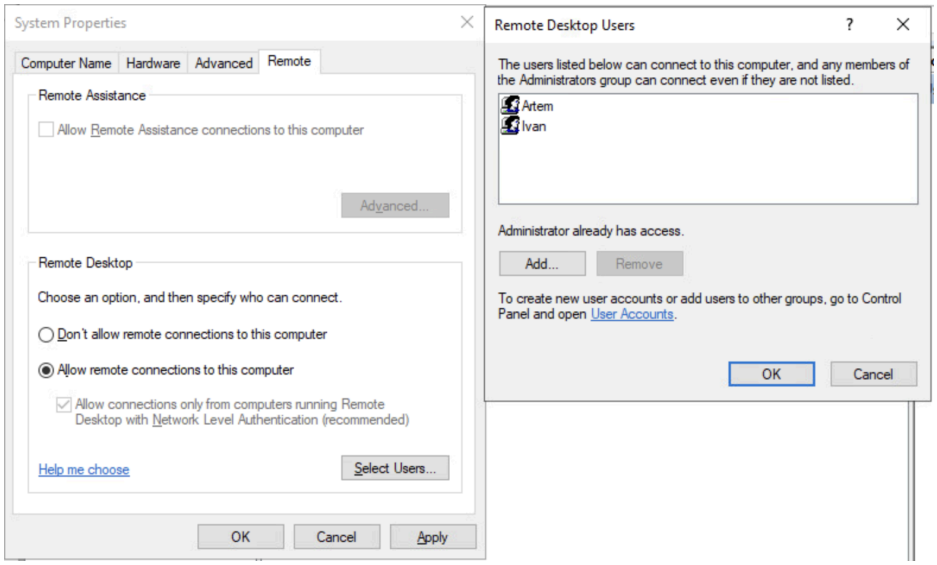


Рисунок 13 – Разрешение пользователям выполнять вход через RDP

Также хорошей практикой является сокрытие пользователей на экране блокировки (рисунок 14).

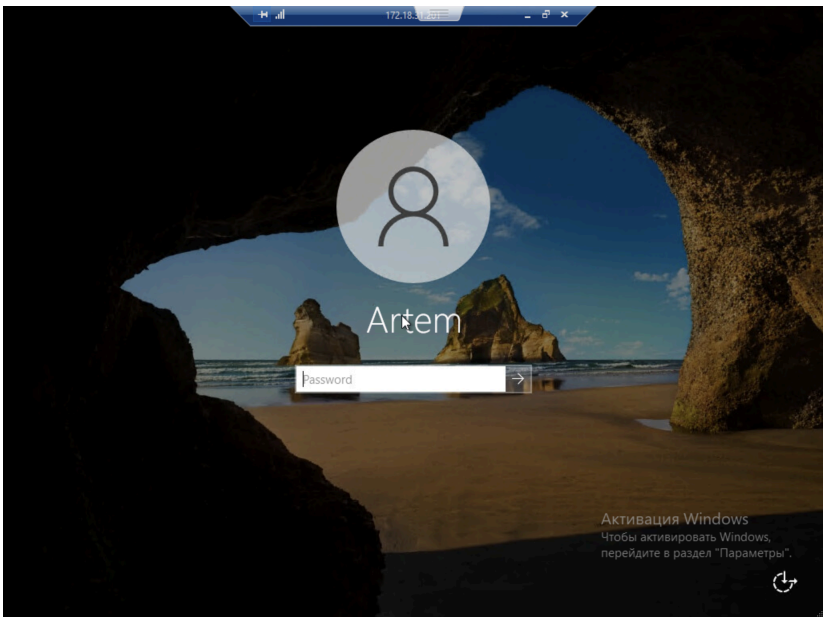


Рисунок 14 – Экран блокировки раскрывает вашего пользователя

Перейдем в уже рассматриваемый ранее редактор групповых политик и перейдем в Конфигурация компьютера → Конфигурация Windows → Параметры безопасности → Локальные политики → Параметры безопасности. Выберем “Интерактивный вход в систему: Не отображать учетные данные последнего пользователя”, отметим “Не отображать пользовательскую информацию” и вновь вернемся на экран блокировки. Теперь имя пользователя не раскрывается (рисунок 15).

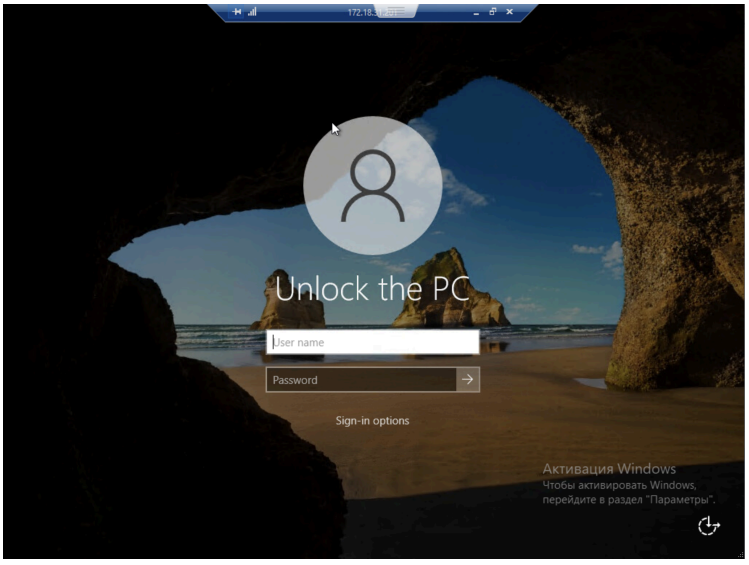


Рисунок 15 – Экран блокировки при сокрытии данных пользователя

Вернемся в нашего пользователя. Откройте блокнот и введите туда любой текст (например, secret). Скопируйте этот текст и попробуйте вставить в блокнот на хостовой системе (требуется скрыть окно RDP). Чтобы убрать у пользователей эту возможность зайдем в оснастку управления групповыми политиками и перейдем в Конфигурация компьютера → Административные шаблоны → Компоненты Windows → Службы удалённых рабочих столов → Узел сеансов удалённых рабочих столов → Перенаправление устройств и ресурсов. Здесь, в идеале, требуется запретить все. Для примера ограничим общий буфер (рисунок 16).

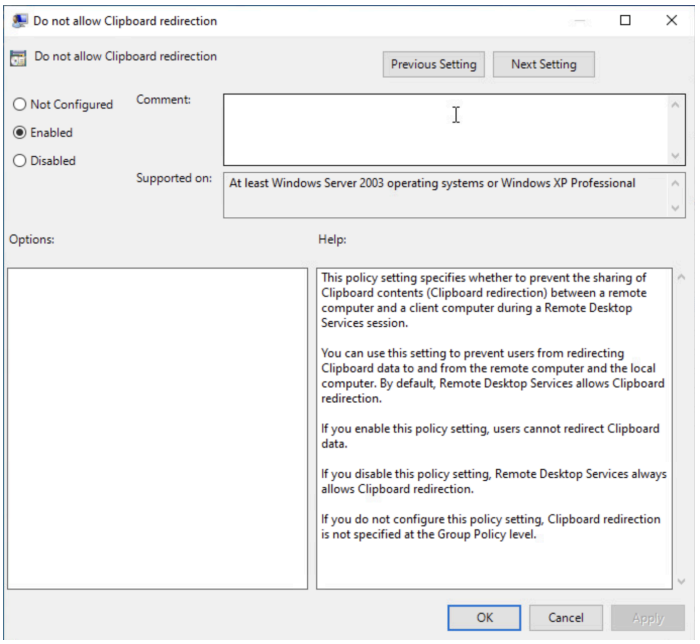


Рисунок 16 – Ограничение общего буфера обмена

После этого общий буфер будет недоступен. Еще раз обратим внимание на то, что в оснастке редактирования групповых политик также присутствует возможность ограничить перенаправление USB-устройств, принтеров, дисков и т.д. Все это является элементами базовой настройки безопасности Windows.

Также важной является возможность оптимизации канала связи и визуализации у сессий RDP. Например, в приложении редактирования

групповых политик в Конфигурация компьютера → Административные шаблоны → Компоненты Windows → Службы удаленных рабочих столов → Узел сеансов удаленных рабочих столов → Среда удаленного сеанса можно отключить ряд анимаций и выставить прямую зависимость между графикой и скоростью соединения “Установить оптимизацию графики для RDP” → “Оптимизировать для низкой скорости”. На рисунке 17 представлен RDP-клиент с включенным параметром “Не показывать обои рабочего стола”.

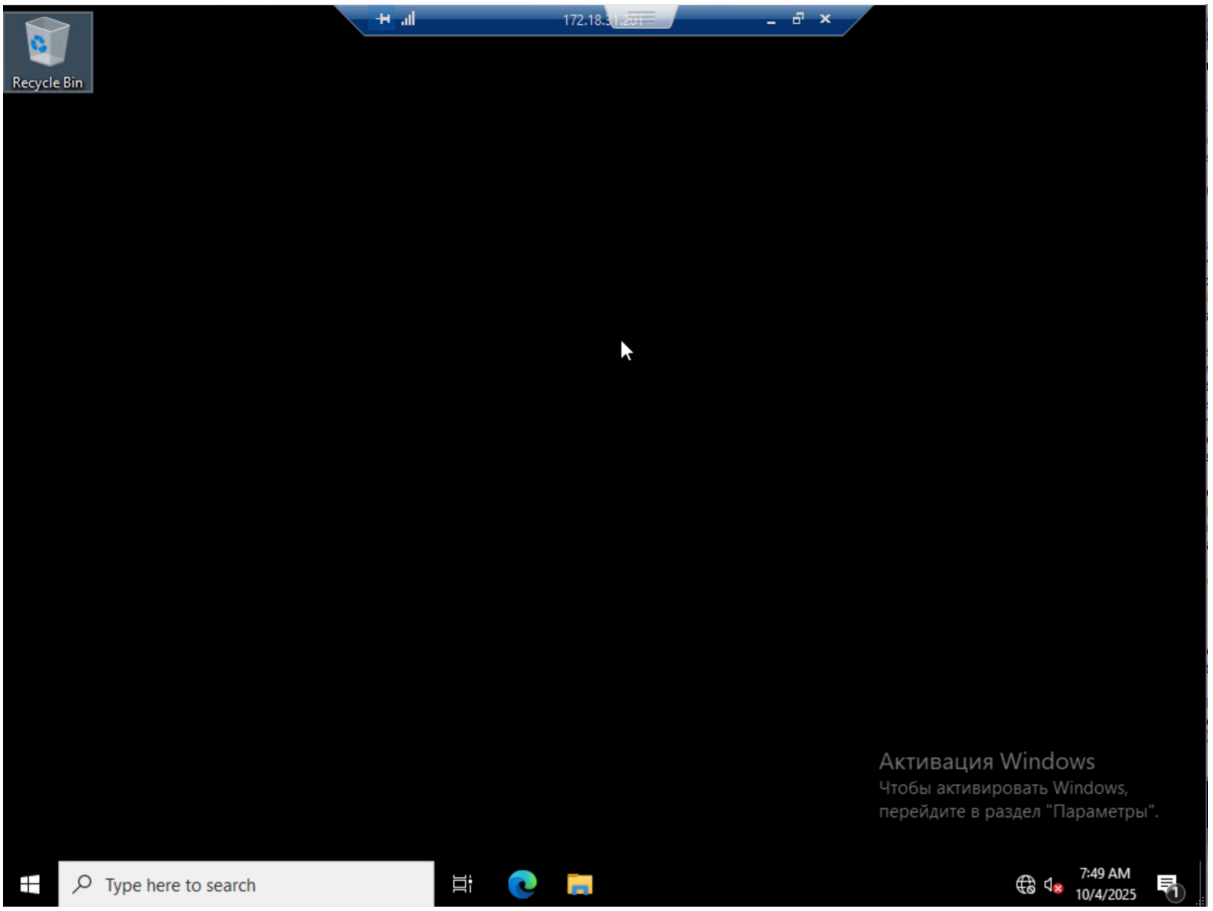


Рисунок 17 – RDP-клиент с включенным параметром “Не показывать обои рабочего стола”

Помимо вышеописанных приемов обеспечения безопасности RDP также предоставляет ряд других функций, с которыми также рекомендуется ознакомиться в той же оснастке gpedit.

2.8 Введение в PKI

PKI (англ. Public Key Infrastructure, инфраструктура открытых ключей) – это совокупность технологий, стандартов, ролей и политик, необходимых для создания, управления, распространения, использования, хранения и отзыва цифровых сертификатов. Иными словами, PKI привносит возможность удостоверять личность того или иного объекта в цепочке доверия и обеспечивает шифрование при обмене данными.

PKI основан на асиметричной криптографии. Ее идея заключается в наличии двух ключей: публичного (public) и приватного (private). В PKI возможны два основных сценария: данные шифруются публичным ключом, чтобы расшифровать их мог только владелец приватного; или наоборот — данные подписываются приватным ключом, чтобы любой, имеющий публичный ключ, мог проверить подлинность. Как и вся криптография, PKI работает корректно пока секрет (приватный ключ) не опубличивается.

В терминологии PKI также существуют сертификат, связывающий публичный ключ и субъект (человека, сервер и т.д.) и центр сертификации (ЦА) – субъект взаимодействия, на доверии к которому все и строится, подписывающий сертификаты;

Рассмотрим типовую схему работы такой инфраструктуры на примере HTTPS. При открытии сайта ваш браузер запрашивает сертификат. После этого сайт возвращает свой сертификат с открытым ключом, подписанный ЦА. Далее проверяется цепочка доверия – мы доверяем нашему ЦА, поскольку ему доверяет вышестоящему ЦА и так до корневого ЦА. Когда браузер удостоверился, что сайту можно доверять, он генерирует симметричный ключ, шифрует его открытым ключом сервера и передает ему для дешифровки. Все последующее взаимодействие шифруется именно им.

2.9 Практические основы VPN

VPN (англ. Virtual Private Network, виртуальная частная сеть) – технология, позволяющая объединить в общую сеть распределенные вычислительные ресурсы. Так, благодаря ней региональные офисы компании фактически находятся в той же локальной сети, что и главный офис. Это полезно не только для простого управления инфраструктурой, но и для обеспечения безопасности.

В рамках курса рассмотрим настройку наиболее популярных систем организации виртуальных частных сетей: openvpn, wireguard, xray.

Начнем с установки и конфигурирования openvpn. Для этого установим пакеты на наш Linux-сервер:

```
sudo apt install openvpn easy-rsa
```

Первый потребуется непосредственно для развертывания сервера, а второй – для генерации сертификатов и ключей. При установке пакета easy-rsa в директории `/usr/share/easy-rsa` появились конфигурационные файлы, которые требуется перенести в нашу новую директорию для openvpn:

```
sudo mkdir /etc/openvpn/easy-rsa
sudo cp -r /usr/share/easy-rsa /etc/openvpn/
```

Изучите содержимое директории `/etc/openvpn/easy-rsa`. После этого выполните лежащий в ней скрипт для создания ЦА на базе нашей системы:

```
sudo ./easyrsa init-pki
```

Создадим наш корневой сертификат и сертификаты для openvpn сервера с использованием команд:

```
sudo ./easyrsa build-ca nopass
sudo ./easyrsa gen-dh
sudo ./easyrsa gen-req server nopass
sudo ./easyrsa sign-req server server
```

```
sudo openvpn --genkey > ta.key
```

После этого перенесем все сгенерированные сертификаты в директорию /etc/openvpn/easy-rsa/pki. Скопируем конфигурационный файл сервера из шаблона:

```
sudo cp pki/ca.crt ta.key pki/issued/server.crt
pki/private/server.key pki/dh.pem ..
sudo cp
/usr/share/doc/openvpn/examples/sample-config-files/
server.conf /etc/openvpn/
```

В нем требуется изменить строку `dh dh2048.pem` на `dh dh.pem` и раскомментировать строку `tls-auth ta.key 0`. Далее самостоятельно запустим сервер:

```
sudo openvpn /etc/openvpn/server.conf
```

Если последним выводом является `Initialization Sequence Completed`, то сервер настроен корректно. Для того, чтобы понять из чего состоит конфигурация клиента, соберем ее самостоятельно. На втором хосте создадим конфигурационный файл client.ovpn. Внутрь сначала запишем стандартную конфигурацию:

```
client
dev tun
proto udp
remote my-server-1 1194
resolv-retry infinite
nobind
persist-key
persist-tun
ca ca.crt
cert client.crt
key client.key
```

```
tls-auth ta.key 1
remote-cert-tls server
verb 3
```

Вместо запишем `my-server-1` IP-адрес сервера. Далее требуется самостоятельно добавить в конец файла теги:

1) <ca></ca>, в который записать корневой сертификат;

2) <cert></cert>, в который записать client.crt из директории /etc/openvpn/easy-rsa/pki/issued/;

3) <key></key>, куда занести информацию из client.key, который хранится в /etc/openvpn/easy-rsa/pki/private/;

4) <tls-auth></tls-auth>, в который записать информацию из ta.key.

Стоит отметить, что также допустимо положить данные файлы в директорию client.ovpn. После этого выполните:

```
sudo openvpn –config client.ovpn
```

Если все сделано правильно, то в на экране у клиента выводится `Initialization Sequence Completed`.

Рассмотрим установку сервера Wireguard. Wireguard – современный VPN-протокол, отличительной особенностью которого является высочайшая производительность. Она достигается за счет легковесной кодовой базы (протокол интегрирован непосредственно в ядро Linux) и использования строгих, современных криптографических примитивов (ChaCha20, Poly1305, Curve25519) вместо поддержки десятков устаревших алгоритмов. Это значительно уменьшает поверхность атаки и упрощает аудит безопасности. Установим пакет wireguard:

```
sudo apt install wireguard
```

Перейдем в директорию с сервером и сгенерируем приватный и публичный ключи:

```
cd /etc/wireguard/
```

```
wg genkey | tee /etc/wireguard/server_privatekey |
wg pubkey | tee /etc/wireguard/server_publickey
```

Создадим конфигурационный файл сервера wg0.conf и запишем:

```
[Interface]
PrivateKey = <ПРИВАТНЫЙ КЛЮЧ>
Address = 8.0.2.0/24
ListenPort = 51820
PostUp = iptables -A FORWARD -i %i -j ACCEPT;
iptables -t nat -A POSTROUTING -o <ИНТЕРФЕЙС> -j
MASQUERADE
PostDown = iptables -D FORWARD -i %i -j ACCEPT;
iptables -t nat -D POSTROUTING -o <ИНТЕРФЕЙС> -j
MASQUERADE
```

Далее создадим клиентскую конфигурацию, выполнив:

wg genkey | tee /etc/wireguard/client_privatekey | wg pubkey | tee /etc/wireguard/client_publickey

И обновим конфигурацию сервера, добавив:

```
[Peer]
PublicKey = <ПУБЛИЧНЫЙ_КЛЮЧ_КЛИЕНТА>
AllowedIPs = 8.0.2.8
```

Запустим сервис:

```
systemctl start wg-quick@wg0
```

На клиентской машине создадим wg.conf:

```
[Interface]
PrivateKey = <ЗАКРЫТЫЙ_КЛЮЧ_КЛИЕНТА>
Address = 8.0.2.8
DNS = 8.8.8.8

[Peer]
```

```
PublicKey = <ПУБЛИЧНЫЙ_КЛЮЧ_СЕРВЕРА>
Endpoint = <АДРЕС_СЕРВЕРА>:<ПОРТ>
AllowedIPs = 0.0.0.0/0
PersistentKeepalive = 20
```

И, установив на ваше устройство клиент, подгрузим конфигурационный файл и нажмем “Activate”. Если все сделано верно, соединение установится (рисунок 18).

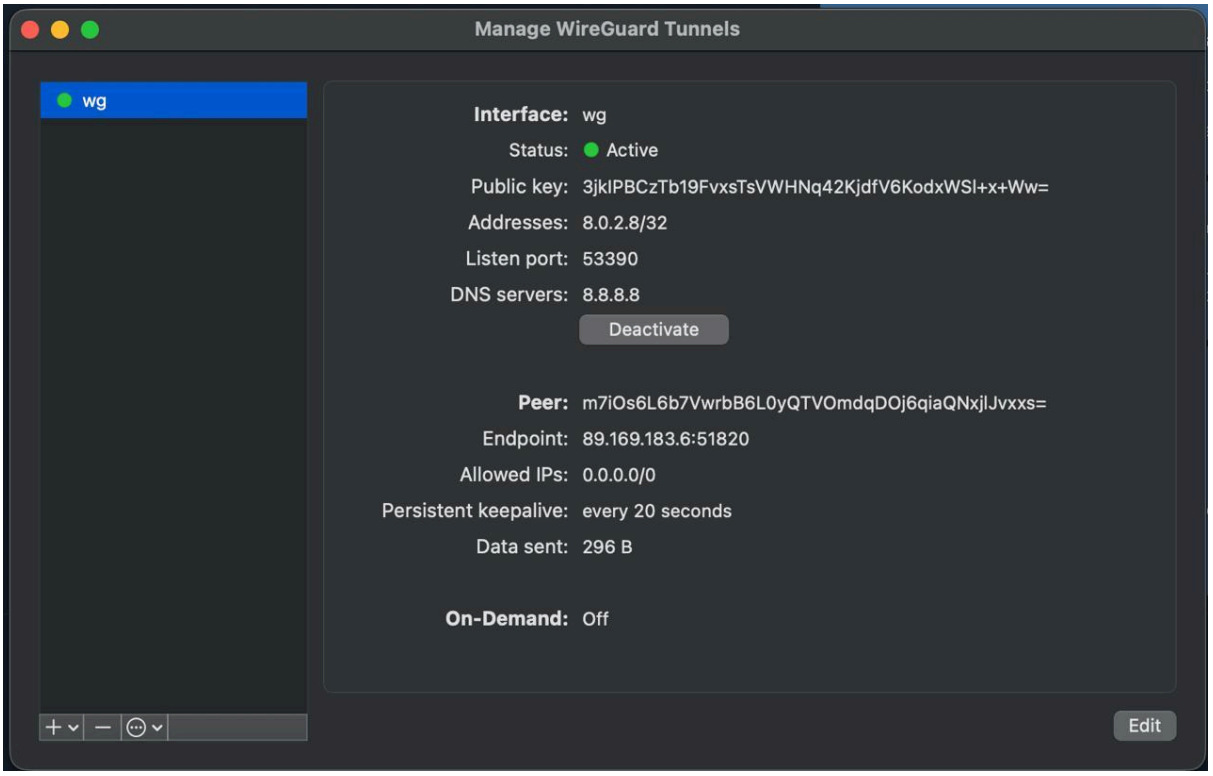


Рисунок 18 – Интерфейс клиента Wireguard на MacOS

Еще одним часто используемым протоколом является Xray. Он основан на V2Ray, но отличается улучшенной производительностью, новыми функциями и более высоким уровнем безопасности. Для начала скачаем утилиту для генерации universally unique id – будущего пароля для Xray:

```
apt install uuid -y
uuid -v 4
```

Сгенерируем самоподписанные сертификаты:

```
sudo mkdir -p /etc/xray/cert
cd /etc/xray/cert
sudo openssl req -newkey rsa:2048 -x509 -sha256
-days 365 -nodes \
  -keyout private.key -out certificate.crt \
  -subj "/CN=example.com"
```

С помощью скрипта установим Xray на сервер:

```
bash -c "$(curl -L
https://github.com/XTLS/Xray-install/raw/main/instal
l-release.sh)" @ install -u root
```

Проверим статус сервера:

```
systemctl status xray
```

Обновим конфигурационный файл `/usr/local/etc/xray/config.json`, добавим туда следующее:

```
{
  "log": {
    "loglevel": "warning"
  },
  "inbounds": [
    {
      "port": 443,
      "protocol": "vless",
      "settings": {
        "clients": [
          {
            "id": "<ВАШ_UUID>",
            "level": 0,
            "email": "user@example.com"
```

```
          }
        ],
        "decryption": "none"
      },
      "streamSettings": {
        "network": "tcp",
        "security": "tls",
        "tlsSettings": {
          "certificates": [
            {
              "certificateFile":
"/etc/xray/cert/certificate.crt",
              "keyFile":
"/etc/xray/cert/private.key"
            }
          ]
        }
      }
    }
  ],
  "outbounds": [
    {
      "protocol": "freedom",
      "settings": {}
    }
  ]
}
```

Единственное что нужно изменить – UUID на ранее сгенерированный. После этого требуется перезагрузить сервис. В нашем случае конфигурация клиента будет выглядеть следующим образом:

```
{
  "outbounds": [
    {
      "protocol": "vless",
      "settings": {
        "vnext": [
          {
            "address": "ВАШ_IP",
            "port": 443,
            "users": [
              {
                "id": "ВАШ_UUID",
                "encryption": "none"
              }
            ]
          }
        ]
      },
      "streamSettings": {
        "network": "tcp",
        "security": "tls",
        "tlsSettings": {
          "serverName": "example.com",
          "allowInsecure": true
        }
      }
```

```
      }
    ]
}
```

Подключение также можно выполнить через один из многих клиентов. Например, на рисунке 19 показан клиент для MacOS.

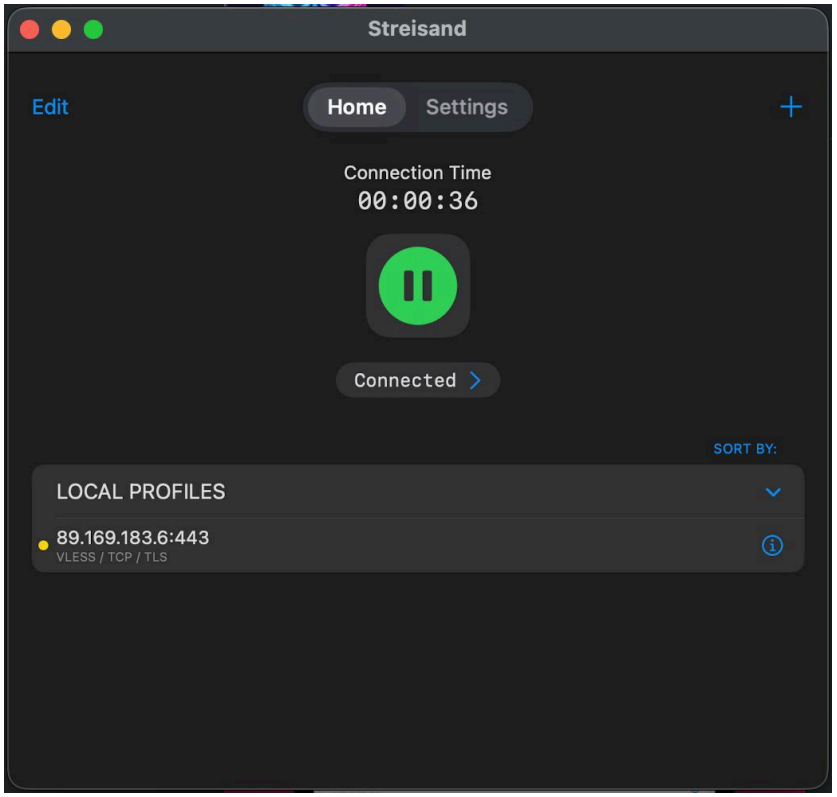


Рисунок 19 – Клиент Xray для MacOS

В свою очередь на сервере в журнале событий будет примерно следующее:

```
root@vpn:/etc/xray/cert# sudo journalctl -u xray -f
Oct 04 20:44:47 vpn xray[6257]: 2025/10/04
20:44:47.810507 from 93.158.190.188:55770 accepted
```

```
tcp:[2a01:b740:1361:101::2:7]:443                email:
user@example.com
Oct   04   20:44:48   vpn   xray[6257]:   2025/10/04
20:44:48.245741  from  93.158.190.188:55772  accepted
tcp:17.242.184.19:443 email: user@example.com
Oct   04   20:44:48   vpn   xray[6257]:   2025/10/04
20:44:48.261557  from  93.158.190.188:55777  accepted
tcp:79.133.169.118:80 email: user@example.com
```

Практическое задание

Необходимо в ранее спроектированную корпоративную вычислительную сеть головного офиса интегрировать межсетевой экран (шлюз). В сравнительном анализе аргументировать выбор не только семейства операционных систем, но и конкретного продукта. Ключевые параметры сравнения: надежность/отказоустойчивость, информационная безопасность, быстродействие, трудоемкость сопровождения.

При первоначальном конфигурировании сервера стоит обратить внимание на следующие аспекты:

1. резервное копирование и масштабируемость;
2. организация и защита удаленного доступа (протоколы, технологии и алгоритмы аутентификации и др.);
3. настройка межсетевого экрана (включая все разновидности *NAT* и *DMZ*);
4. устранение опубликованных уязвимостей и известных инструментов скрытого управления компьютером (бэкдоров);
5. установка дополнительных средств защиты;
6. реализация дополнительных заданий по вариантам.

Вариант 1

Разверните две виртуальные машины: Ubuntu Server (межсетевой экран и VPN-сервер) и Windows Server. Настройте на Ubuntu базовую конфигурацию iptables с NAT, обеспечив выход Windows в интернет через Ubuntu. Разверните XrayVPN и настройте подключение извне по защищённому RDP к Windows через VPN-туннель. Добавьте фильтрацию IP-адресов, разрешая доступ только из России.

Вариант 2

Разверните две виртуальные машины: Ubuntu Server (межсетевой экран и VPN-сервер) и Windows Server. Настройте на Ubuntu базовую конфигурацию iptables с NAT, обеспечив выход Windows в интернет через Ubuntu. Разверните OpenVPN-сервер и настройте подключение извне по защищённому RDP к Windows через VPN-туннель. Добавьте фильтрацию IP-адресов, разрешая доступ только из России.

Вариант 3

Разверните две виртуальные машины: Ubuntu Server (межсетевой экран и VPN-сервер) и Windows Server. Настройте на Ubuntu базовую конфигурацию iptables с NAT, обеспечив выход Windows в интернет через Ubuntu. Разверните WireGuard-сервер и настройте подключение извне по защищённому RDP к Windows через VPN-туннель. Включите в iptables TARPIT для защиты от активного сканирования портов.

Вариант 4

Разверните две виртуальные машины: Ubuntu Server (межсетевой экран и VPN-сервер) и Windows Server. Настройте на Ubuntu базовую конфигурацию iptables с NAT, обеспечив выход Windows в интернет через Ubuntu. Разверните XrayVPN и настройте подключение извне по защищённому RDP к Windows через VPN-туннель. В Windows настройте RDP с обязательным использованием TLS-шифрования, а на Ubuntu добавьте DELUDE-правила для имитации “фильтруемых” портов.

Вариант 5

Разверните две виртуальные машины: Ubuntu Server (межсетевой экран и VPN-сервер) и Windows Server. Настройте на Ubuntu базовую конфигурацию iptables с NAT, обеспечив выход Windows в интернет через

Ubuntu. Разверните OpenVPN-сервер и настройте подключение извне по защищённому RDP к Windows через VPN-туннель. В Windows включите сокрытие пользователей на экране входа и запрет перенаправления буфера обмена. На Ubuntu добавьте блокировку всех SSH-подключений извне, кроме доверенных IP.

Вариант 6

Разверните две виртуальные машины: Ubuntu Server (межсетевой экран и VPN-сервер) и Windows Server. Настройте на Ubuntu базовую конфигурацию iptables с NAT, обеспечив выход Windows в интернет через Ubuntu. Разверните WireGuard-сервер и настройте подключение извне по защищённому RDP к Windows через VPN-туннель. Реализуйте защиту от сканирования портов с помощью xtables-addons и включите NLA (Network Level Authentication) в Windows.

Вариант 7

Разверните две виртуальные машины: Ubuntu Server (межсетевой экран и VPN-сервер) и Windows Server. Настройте на Ubuntu базовую конфигурацию iptables с NAT, обеспечив выход Windows в интернет через Ubuntu. Разверните OpenVPN-сервер и настройте подключение извне по защищённому RDP к Windows через VPN-туннель. В iptables реализуйте замедление RDP-соединений при множественных попытках подключения.

Вариант 8

Разверните две виртуальные машины: Ubuntu Server (межсетевой экран и VPN-сервер) и Windows Server. Настройте на Ubuntu базовую конфигурацию iptables с NAT, обеспечив выход Windows в интернет через Ubuntu. Разверните XrayVPN и настройте подключение извне по защищённому RDP к Windows через VPN-туннель. На Windows

оптимизируйте канал RDP, отключив анимации и визуальные эффекты. В iptables добавьте фильтрацию входящих соединений по геолокации (разрешить только РФ).

Вариант 9

Разверните две виртуальные машины: Ubuntu Server (межсетевой экран и VPN-сервер) и Windows Server. Настройте на Ubuntu базовую конфигурацию iptables с NAT, обеспечив выход Windows в интернет через Ubuntu. Разверните WireGuard-сервер и настройте подключение извне по защищённому RDP к Windows через VPN-туннель. Настройте TARPIT для замедления агрессивных сканирований и ограничьте SSH по времени и количеству подключений.

Вариант 10

Разверните две виртуальные машины: Ubuntu Server (межсетевой экран и VPN-сервер) и Windows Server. Настройте на Ubuntu базовую конфигурацию iptables с NAT, обеспечив выход Windows в интернет через Ubuntu. Разверните OpenVPN-сервер и настройте подключение извне по защищённому RDP к Windows через VPN-туннель. В Windows настройте полную изоляцию RDP: запрет перенаправления устройств, буфера обмена и отображения пользователей. В iptables добавьте защиту от сканирования.

Содержание отчета

1. Цель работы.
2. Техническое задание.
3. Сравнительный анализ существующих решений.
4. Установка и конфигурирование серверного решения.
5. Реализация комплекса мер по обеспечению ИБ.
6. Заключение.